



# PROJECT HANDBOOK

## A Plain-Language Guide & Tutorial to the BrandPulse AI Sentiment-Analysis Project

This handbook is a companion to the internship report. It explains, in plain language, what each part of the project does and why, walks you step-by-step through setting up and running everything yourself, and prepares you to discuss the project confidently in a viva or interview. Every number here matches the report and notebooks — there is no new analysis.

Aaditiya Anjan · Chandigarh University · Persevex AI Internship · 2026

# How to Use This Handbook

---

Read it top to bottom once, then use it as a quick reference. The handbook has three kinds of content:

- **Explanations** — plain-language descriptions of every project component (Sections 1–6).
- **Tutorials** — copy-paste commands to set up the environment, reproduce the models, and launch the dashboard (Section 7).
- **Viva preparation** — anticipated questions with strong, concise answers, plus a glossary of every technical term used (Sections 8–9).

# 1. The Big Picture

---

A brand's reputation lives or dies on social media. Thousands of people tweet about an airline every day — some happy, most not — and no human team can read all of it. BrandPulse AI reads each tweet automatically and labels it Positive, Neutral, or Negative, so a marketing team can see the mood of their customers at a glance and watch it change over time.

## Q: Why does this matter to a business?

Because a single viral complaint can cost millions in reputation. If you can measure sentiment in real time, you can spot a problem early and respond before it spreads.

## Q: What makes tweets so hard to analyse?

They are the messiest text there is: slang, sarcasm, emojis, misspellings, @mentions, #hashtags, links, and negation (“not good” means the opposite of “good”). A naive keyword counter gets fooled constantly — which is exactly why we built a proper cleaning pipeline and two different model families.

## Q: What is the one-sentence summary of the project?

We classify airline tweets into three sentiment classes, compare a classical TF-IDF model against a deep-learning Bi-LSTM, evaluate them honestly on imbalanced data, and serve both through a live dashboard.

# 2. The Data

---

We used the **Twitter US Airline Sentiment** dataset — 14,640 real tweets about US airlines from February 2015, each hand-labelled by humans as positive, neutral, or negative, and each with a real timestamp.

## Q: Why this dataset and not Sentiment140 (the 1.6M-tweet one)?

Because the brief asks for THREE classes (positive/neutral/negative). Sentiment140 only has two. The airline dataset is natively three-class, and its real timestamps let the dashboard draw a genuine “sentiment over time” trend line.

## Q: What is the catch with this data?

It is heavily imbalanced — about 63% of tweets are negative, 21% neutral, and only 16% positive. People mostly tweet at airlines to complain. This imbalance changes how we measure success (see Section 5).

### 3. Cleaning the Text

Before any model sees a tweet, it goes through one shared cleaning module (`src/preprocess.py`). The same module is used in training AND in the live app, so the model always sees text cleaned the same way.

The pipeline, in order:

- **Lowercase** everything and decode HTML (& → &).
- **Strip noise** with regex: URLs, @mentions, the # in hashtags, numbers, and stretched letters (loooooove → loove).
- **Expand contractions** (can't → can not) so negation survives.
- **Tokenise** and **lemmatise** (running → run) using part-of-speech tags.
- **Remove stop-words** (the, a, is...) but KEEP negation words.
- **Fuse negation**: “not good” becomes a single token `not_good`.

#### Q: What is the clever bit here — negation fusion?

A bag-of-words model sees the word “good” inside “not good” and thinks it is positive. By gluing the negation word to the next word (`not_good`), we create one clearly-negative feature. This is our direct answer to the brief's question about sarcasm and negation.

#### Q: Why one shared cleaning file instead of cleaning twice?

To avoid “train/serve skew”. If the app cleaned text even slightly differently from training, accuracy would silently rot. One file = one source of truth.

#### Q: What does lemmatisation do, exactly?

It reduces a word to its dictionary base form using grammar: “running”, “ran” and “runs” all become “run”, so the model treats them as one feature instead of three.

### 4. The Two Models

The whole point of the project is to **compare** two philosophies of NLP.

**Approach A — Classical (the baseline).** We turn text into numbers with **TF-IDF** (which scores how important each word is to a tweet) and feed those numbers to **Logistic Regression** and **Complement Naive Bayes**. Fast, cheap, and fully interpretable — we can read the exact words that drove a decision.

**Approach B — Deep Learning (the advanced model).** We tokenise the text, pad it to 40 words, and feed it to an **Embedding** → **Bi-LSTM** network. Unlike TF-IDF, this reads words *in order*, so it can tell “not happy” from “happy”.

#### Q: What is TF-IDF in one sentence?

Term Frequency–Inverse Document Frequency: a word scores high if it appears a lot in THIS tweet but rarely across ALL tweets — i.e. it is distinctive.

#### Q: What does an LSTM actually do?

It reads the tweet word by word and carries a “memory” of what it has seen, so context from earlier words influences later ones. “Bi” means it reads both forwards and backwards. The Embedding layer first turns each word into a vector that captures meaning.

**Q: What was the macOS deadlock you mention in the report?**

On Apple Silicon, if pandas (which loads pyarrow) is imported before TensorFlow, their threading libraries clash and `model.fit()` freezes at 0% CPU forever. The fix: always import TensorFlow first. Knowing this is a strong real-engineering talking point.

## 5. The Results — and How We Judge Them

| Model                      | Accuracy      | Macro F1      | Weighted F1   |
|----------------------------|---------------|---------------|---------------|
| <b>Logistic Regression</b> | <b>0.7704</b> | <b>0.7233</b> | <b>0.7761</b> |
| Complement Naive Bayes     | 0.7734        | 0.6860        | 0.7603        |
| Bi-LSTM                    | 0.7741        | 0.7153        | 0.7768        |

### Q: Why is the headline number Macro-F1 and not Accuracy?

Because 63% of tweets are negative, a lazy model that guesses “negative” every time would score 63% accuracy while being useless. Macro-F1 averages the score across all three classes equally, so it only rewards a model that also gets the rare positive and neutral tweets right.

### Q: Surprise result — the simple model won?

Yes. Logistic Regression scored the best Macro-F1 (0.7233) — slightly ahead of the Bi-LSTM (0.7153). On a modestly sized, imbalanced dataset, a well-tuned classical model is hard to beat, it trains in seconds, and it explains itself. Deep learning usually pulls ahead only with much more data.

### Q: Where do BOTH models fail?

They confuse neutral and negative most often. A flat complaint like “flight delayed 2 hrs” has no angry words, so it reads as neutral. Sarcasm (“great, another delay”) is the hardest case of all.

### Q: What does precision vs recall mean here?

Precision = of the tweets the model CALLED negative, how many really were. Recall = of the tweets that really WERE negative, how many the model caught. F1 balances the two.

## 6. The Dashboard

Phase 4 is a live Streamlit web app, redesigned in an Apple-inspired light style. You type a tweet and instantly see both models' verdicts side by side, each with a confidence ring and the words that drove the classical decision. Other tabs show the live-stream simulation, the sentiment pie, the real-timestamp trend line, and the full model-comparison center.

### Q: Why does the app flag some predictions “needs human review”?

If the model's confidence is below 60%, forcing a label would be dishonest. Flagging it for a human is realistic product behaviour and shows mature thinking about ML in production.

### Q: What are the four tabs?

Predictor (type a tweet, compare both models), Live Stream (replays tweets as a feed with pie + trend), Model Comparison (KPIs, metric chart, confusion matrices), and Analytics (learning curves, word clouds, training summary).

## 7. Step-by-Step Tutorials

### 7.1 Set up the environment

From the project root, create a virtual environment, install the dependencies, and download the NLTK data the cleaning pipeline needs:

```
python3 -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install -r requirements.txt

python -c "import nltk; [nltk.download(p) for p in [
    'stopwords', 'wordnet', 'omw-1.4', 'punkt', 'punkt_tab',
    'averaged_perceptron_tagger', 'averaged_perceptron_tagger_eng']]"
```

### 7.2 Reproduce the models

Run the two notebooks top-to-bottom (or use the standalone trainer for the LSTM). On macOS, TensorFlow is imported before pandas to avoid the threadpool deadlock.

```
# Phase 1 + 2A – cleaning + classical models
jupyter nbconvert --to notebook --execute --inplace \
    notebooks/01_Preprocessing_and_Classical.ipynb

# Phase 2B – the Bi-LSTM (or: python scripts/train_lstm.py)
jupyter nbconvert --to notebook --execute --inplace \
    notebooks/02_Deep_Learning_LSTM.ipynb

# Phase 3 – regenerate the comparison report + figures
python scripts/make_report.py
```

### 7.3 Launch the dashboard

```
streamlit run dashboard/app.py
# then open the local URL Streamlit prints (usually http://localhost:8501)
```

### 7.4 Predict on your own text (no app)

You can score any sentence directly from Python using the same shared pipeline:

```
import joblib
from src.preprocess import clean_tweet, ID2LABEL

vec = joblib.load('models/tfidf_vectorizer.pkl')
model = joblib.load('models/logreg_model.pkl')

text = 'the crew was wonderful but the delay was awful'
X = vec.transform([clean_tweet(text)])
print(ID2LABEL[model.predict(X)[0]]) # -> negative / neutral / positive
```



## 8. Quick-Fire Viva Preparation

---

### Q: Walk me through your project in 30 seconds.

I built BrandPulse AI, a tweet-sentiment system that classifies airline tweets as positive/neutral/negative. I compared a classical TF-IDF + Logistic Regression pipeline against an Embedding + Bi-LSTM deep model, evaluated both with macro-F1 because the data is imbalanced, and served them through a live Apple-styled Streamlit dashboard.

### Q: What was the hardest part?

Phase 1 — cleaning. Real tweets are chaos. Negation handling and keeping the train and serve cleaning identical were the two decisions that most affected accuracy.

### Q: Why did you use class weighting?

To stop the model from ignoring the rare positive and neutral classes. Class weighting makes mistakes on minority classes “cost” more during training, so the model pays attention to them.

### Q: How do you know the LSTM didn't just overfit?

I monitored validation loss with EarlyStopping and `restore_best_weights`, so the saved model is the best-validation epoch, not the final overfit one. The learning curves show training accuracy rising while validation plateaus — and that is exactly where EarlyStopping halted.

### Q: If you had two more weeks, what next?

Fine-tune a transformer like DistilBERT, add probability calibration, and stream live tweets from the X API behind the dashboard.

### Q: Why does a simpler model beating a neural net make sense?

With only ~12k training tweets and strong class imbalance, the LSTM doesn't have enough data to learn context better than TF-IDF captures keywords. Deep learning's edge shows up at much larger scale.

## 9. Glossary

|                            |                                                                                                                |
|----------------------------|----------------------------------------------------------------------------------------------------------------|
| <b>Sentiment analysis</b>  | Automatically determining the emotional polarity (positive / neutral / negative) of a piece of text.           |
| <b>TF-IDF</b>              | Term Frequency–Inverse Document Frequency. A way to turn text into numbers that highlights important words.    |
| <b>Bag of words</b>        | Representing text as an unordered collection of word counts, ignoring grammar and word order.                  |
| <b>Lemmatisation</b>       | Reducing a word to its dictionary base form (running → run).                                                   |
| <b>Stop-words</b>          | Extremely common words (the, a, is) usually removed because they carry little meaning.                         |
| <b>Logistic Regression</b> | A linear classifier that outputs class probabilities; its coefficients reveal which features drive each class. |
| <b>Naive Bayes</b>         | A probabilistic classifier based on Bayes' theorem; the Complement variant is strong on imbalanced data.       |
| <b>Embedding</b>           | A learned dense vector representation of a word that captures meaning.                                         |
| <b>LSTM</b>                | Long Short-Term Memory — a recurrent neural network cell that remembers long-range context.                    |
| <b>Bidirectional</b>       | Reading a sequence both forwards and backwards so each word sees past and future context.                      |
| <b>Class imbalance</b>     | When some classes appear far more often than others (here, 63% negative), biasing naive models.                |
| <b>Macro-F1</b>            | The F1-score computed per class and then averaged equally — the fair metric for imbalanced data.               |
| <b>Precision / Recall</b>  | Precision: how many predicted-positives were right. Recall: how many real positives were caught.               |
| <b>Confusion matrix</b>    | A grid showing, for each true class, how many items were predicted into each class.                            |
| <b>EarlyStopping</b>       | Halting training when validation performance stops improving, to prevent overfitting.                          |
| <b>Train/serve skew</b>    | When data is processed differently at training and prediction time, silently degrading accuracy.               |

— End of Handbook —